



CS 498: RAG Systems

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

Recall: MoE Models are Sparse with Context-specific Experts

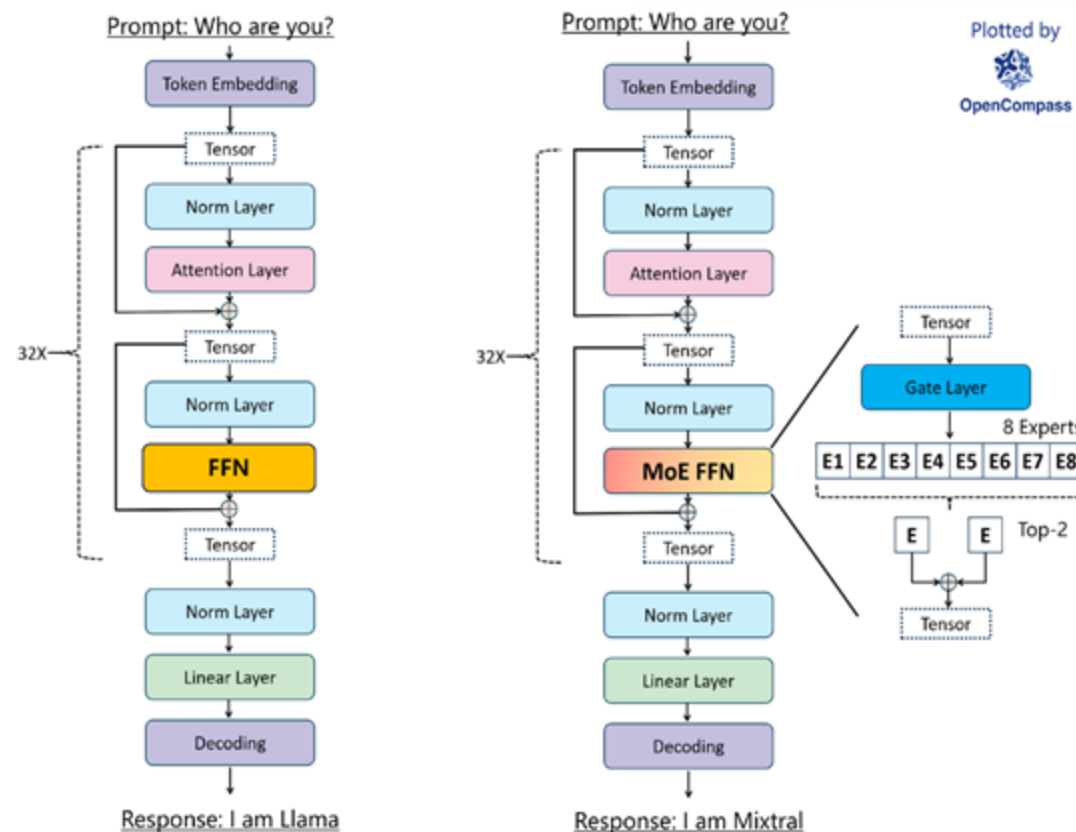


Dense Models:

- All parameters are used in forward and backward paths
- Increasing model capacity needs more computation
- **Larger model size → Higher compute requirements (FLOPs)**

Sparse MoE models

- Sparse utilization of subset of parameters based on input
- Same computation is needed regardless of the model size
- **Larger model size → Similar/Same Compute requirements**



Retrieval-augmented Generation

- Live knowledge distillation without model training/tuning

Objective:

- Understand the motivation behind RAGs
- Optimizing RAG efficiency

- The knowledge is stored implicitly in the parameters of the network.

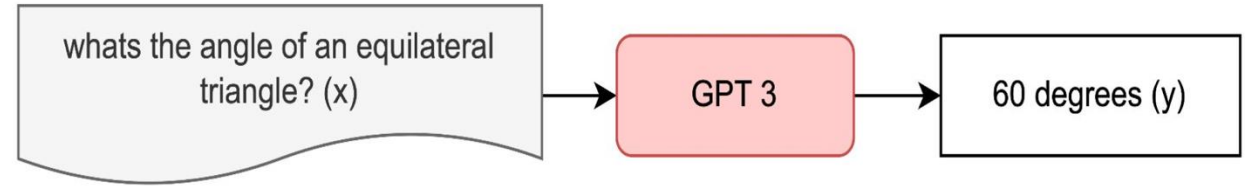


Fig 1: OpenQA Example

- The knowledge is stored implicitly in the parameters of the network.

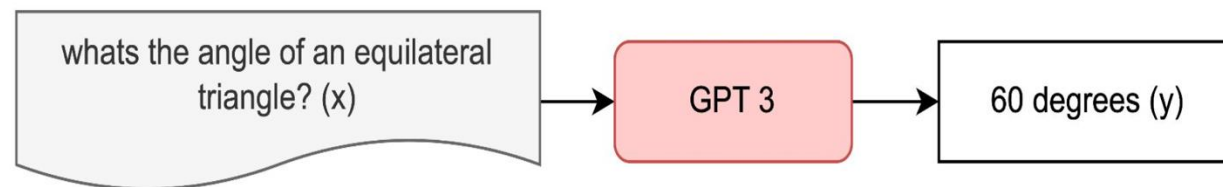


Fig 1: OpenQA Example

- To increase facts/knowledge, the size of the network needs to be increased.

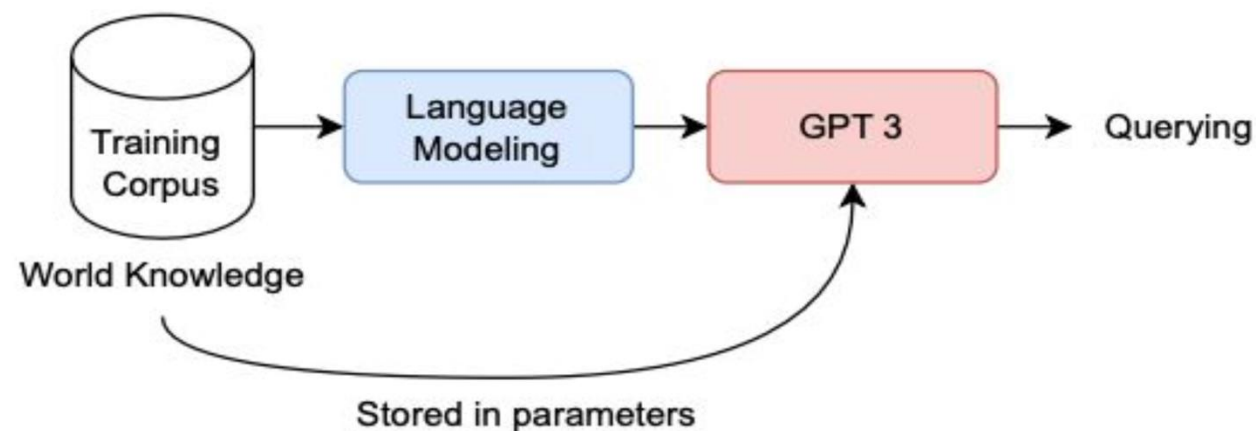


Fig 2: How traditional LMs perform Open QA

OpenQA with Retrieval-augmented Generation (RAG)

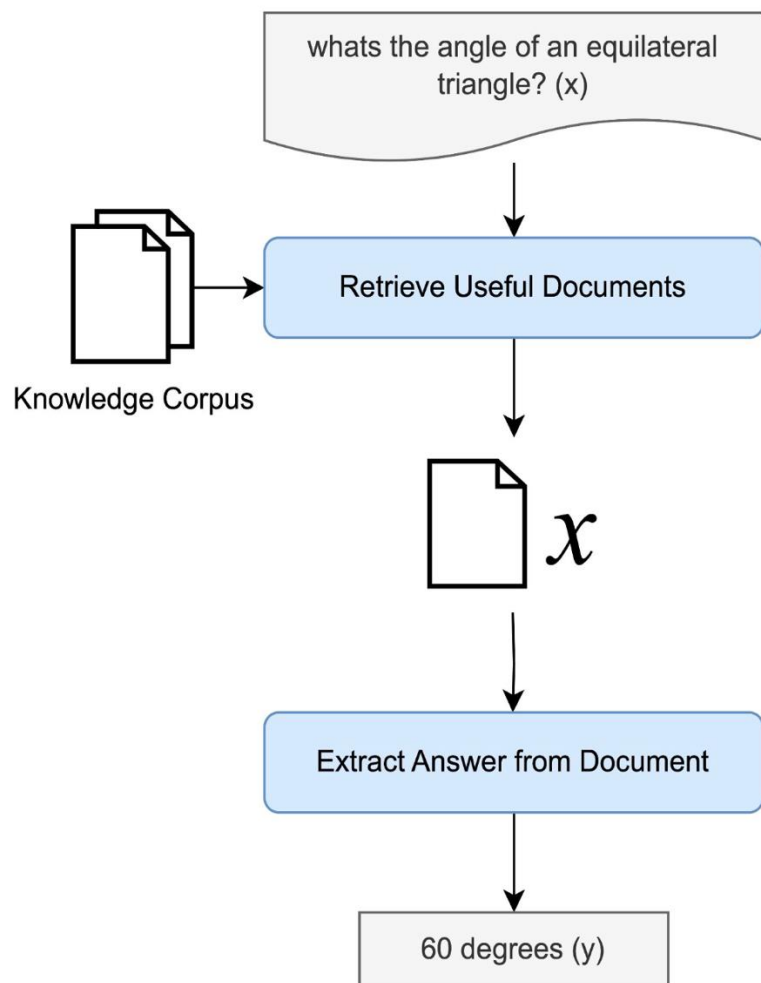
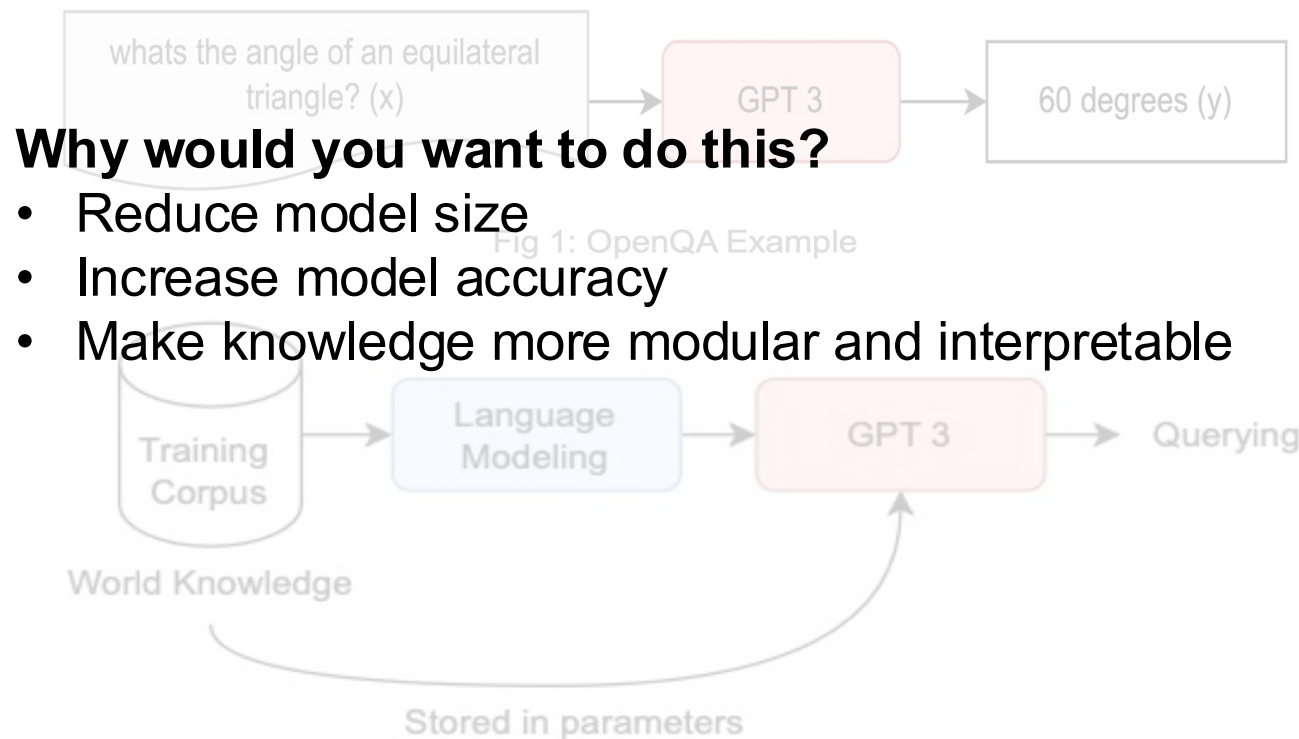


Figure 3: Retrieval Approach Overview



OpenQA with Retrieval-augmented Generation (RAG)

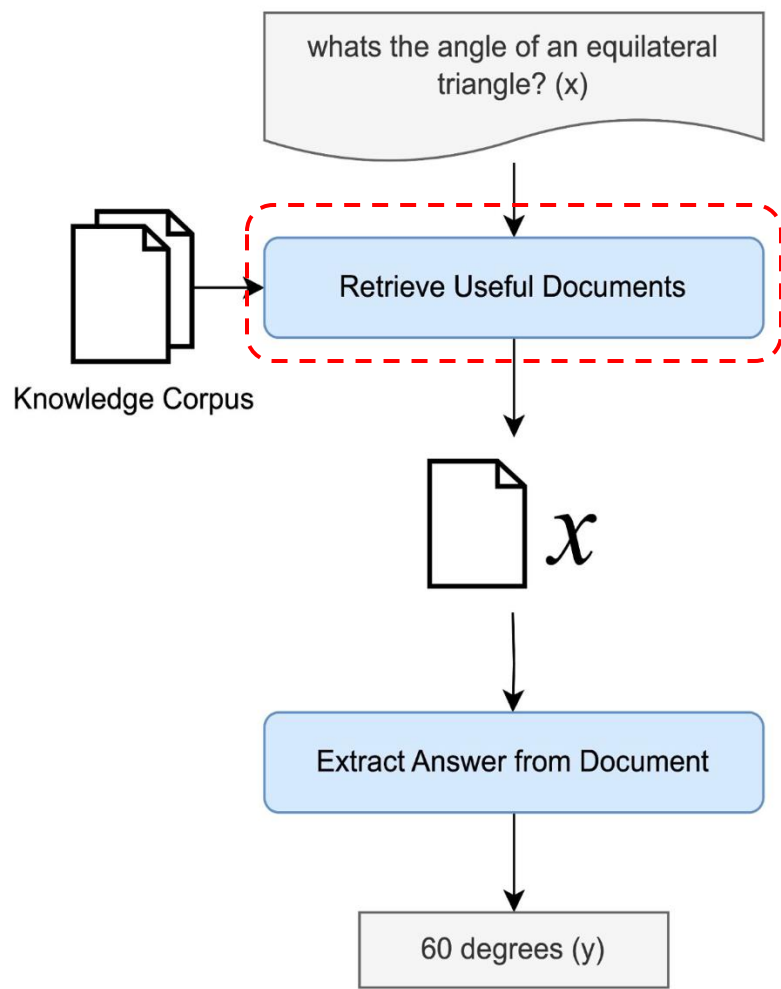


Figure 3: Retrieval Approach Overview

Why would you want to do this?

- Reduce model size
- Increase model accuracy
- Make knowledge more modular and interpretable



Two Important Components:

- A learned model to retrieve documents

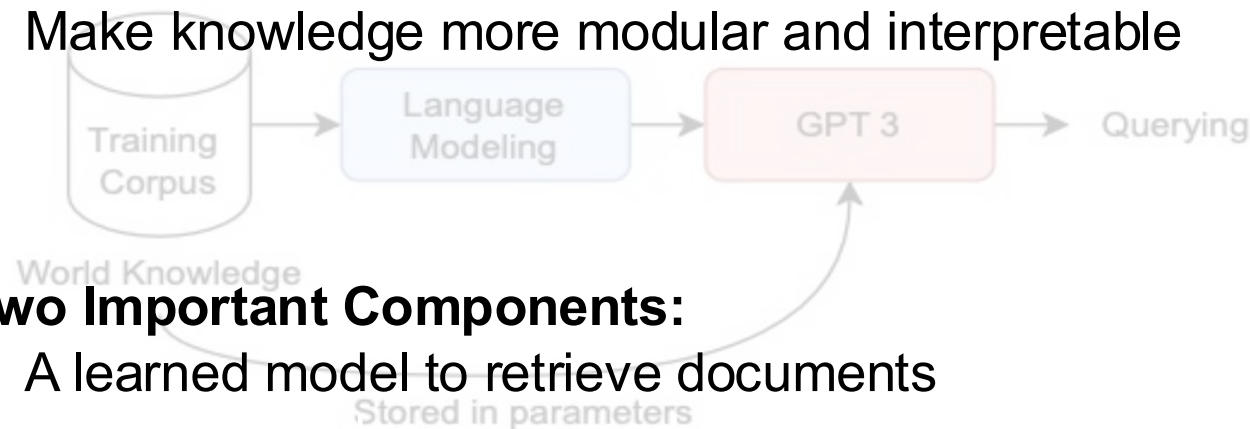


Fig 2: How traditional LMs perform Open QA

OpenQA with Retrieval-augmented Generation (RAG)

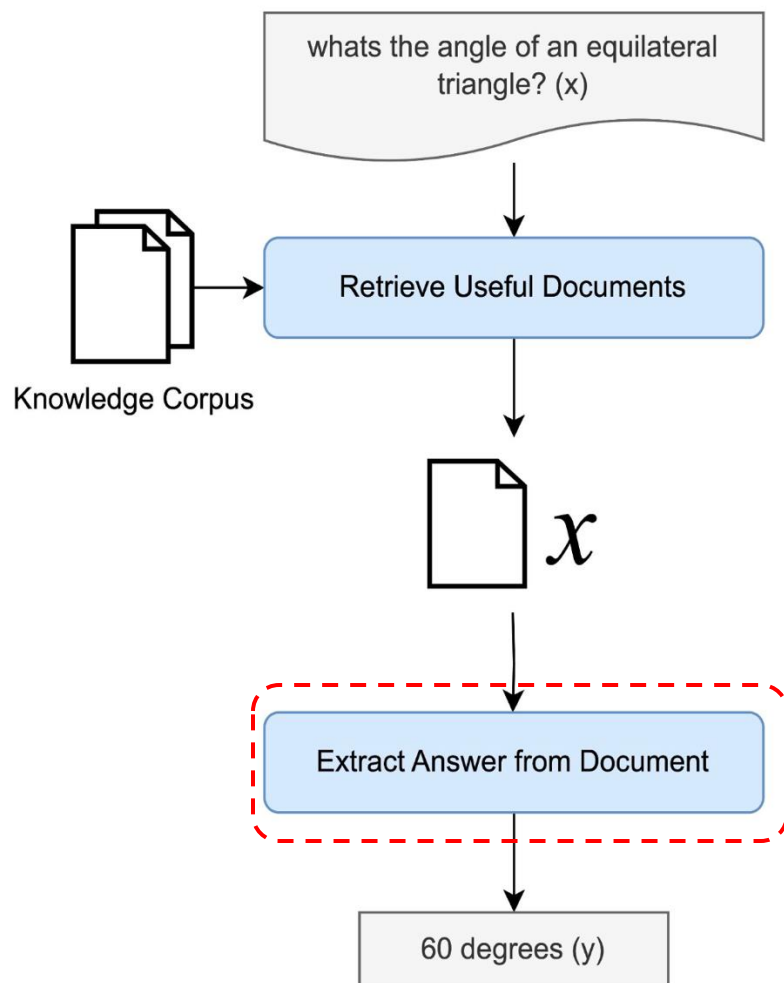


Figure 3: Retrieval Approach Overview

Why would you want to do this?

- Reduce model size
- Increase model accuracy
- Make knowledge more modular and interpretable

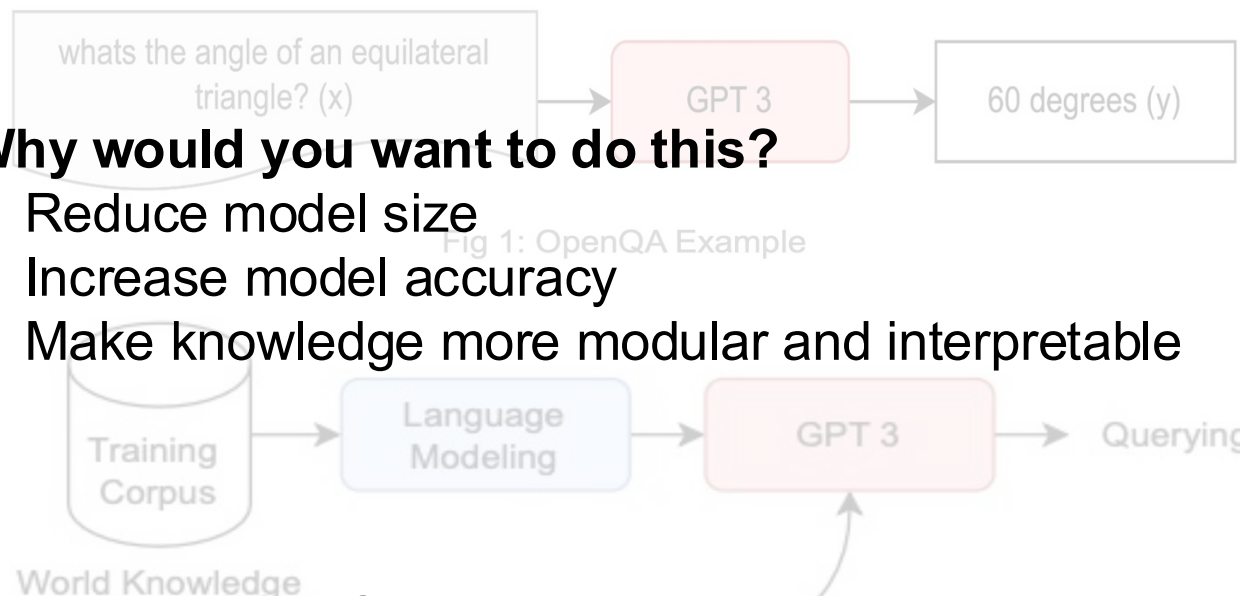


Fig 1: OpenQA Example

Two Important Components:

- A learned model to retrieve documents
- A learned model to answer using documents

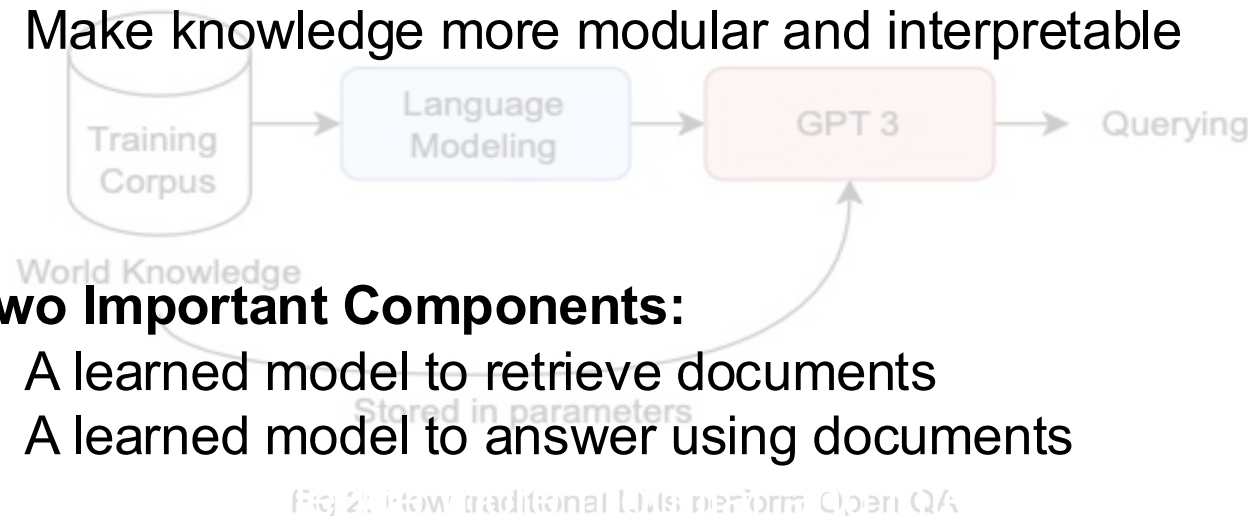
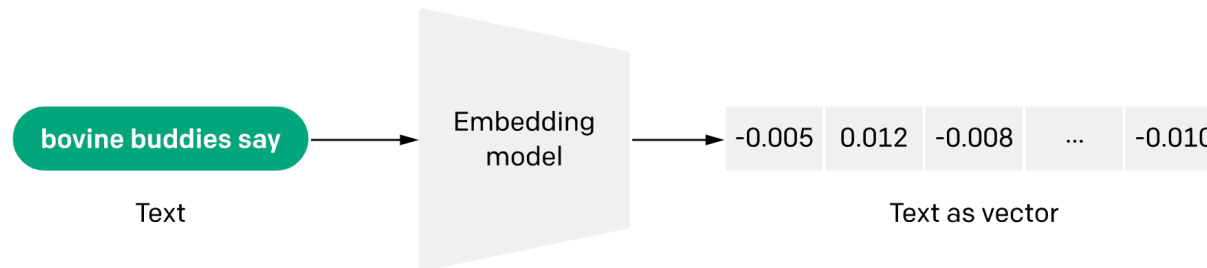


Fig 2: How traditional LLMs perform Open QA

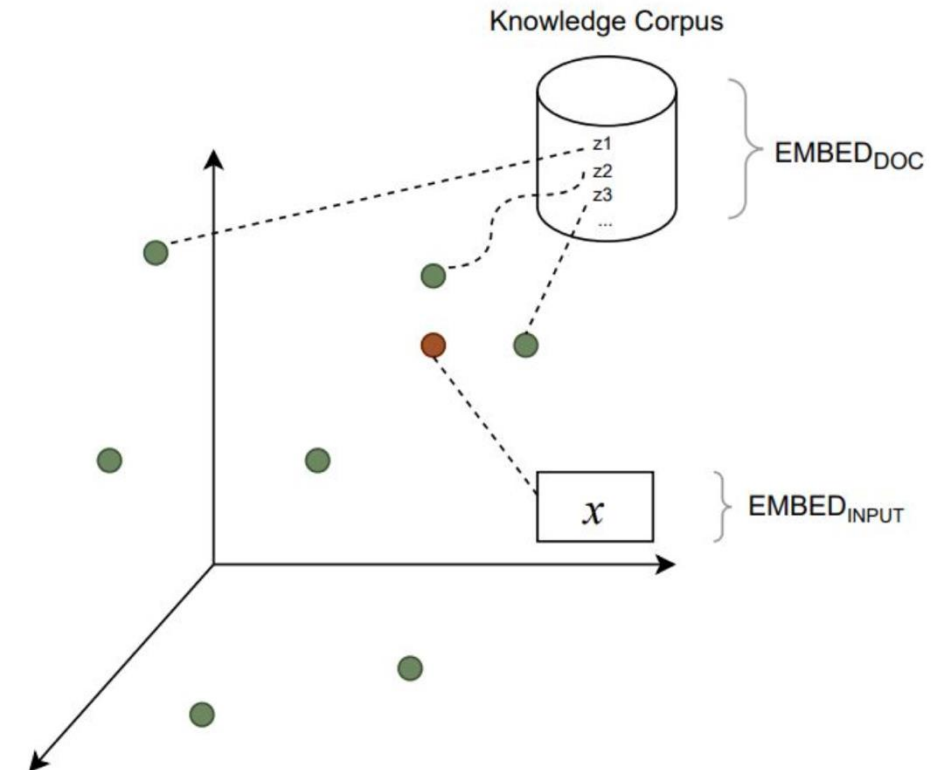
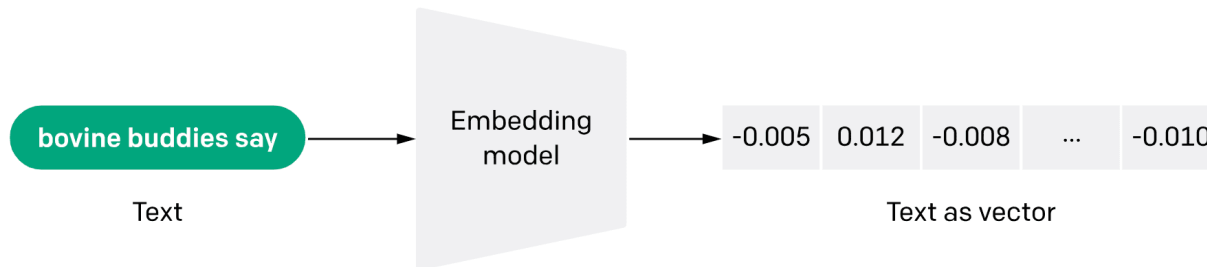
How do we compare an input and documents from the knowledge corpus?

- Embed them into a d-dimensional **vector space** (e.g., BERT model)



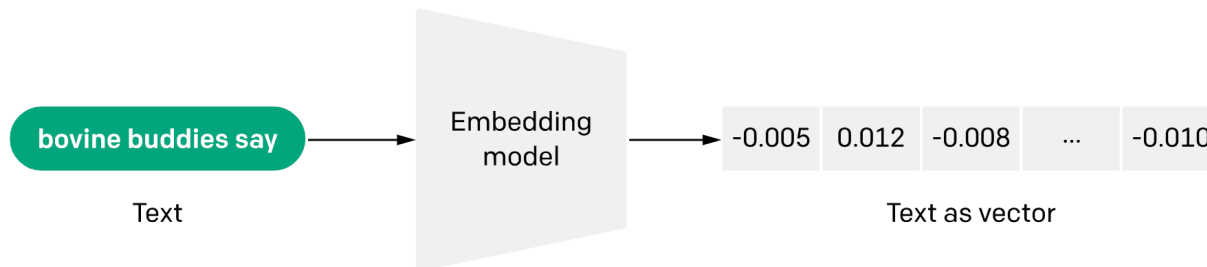
How do we compare an input and documents from the knowledge corpus?

- Embed them into a d-dimensional **vector space** (e.g., BERT model)
- Pick top-k with high $f(x, z) = (\text{EMB}_{\text{input}})^T (\text{EMB}_{\text{doc}})$



How do we compare an input and documents from the knowledge corpus?

- Embed them into a d-dimensional **vector space** (e.g., BERT model)
- Pick top-k with high $f(x, z) = (\text{EMB}_{\text{input}})^T (\text{EMB}_{\text{doc}})$

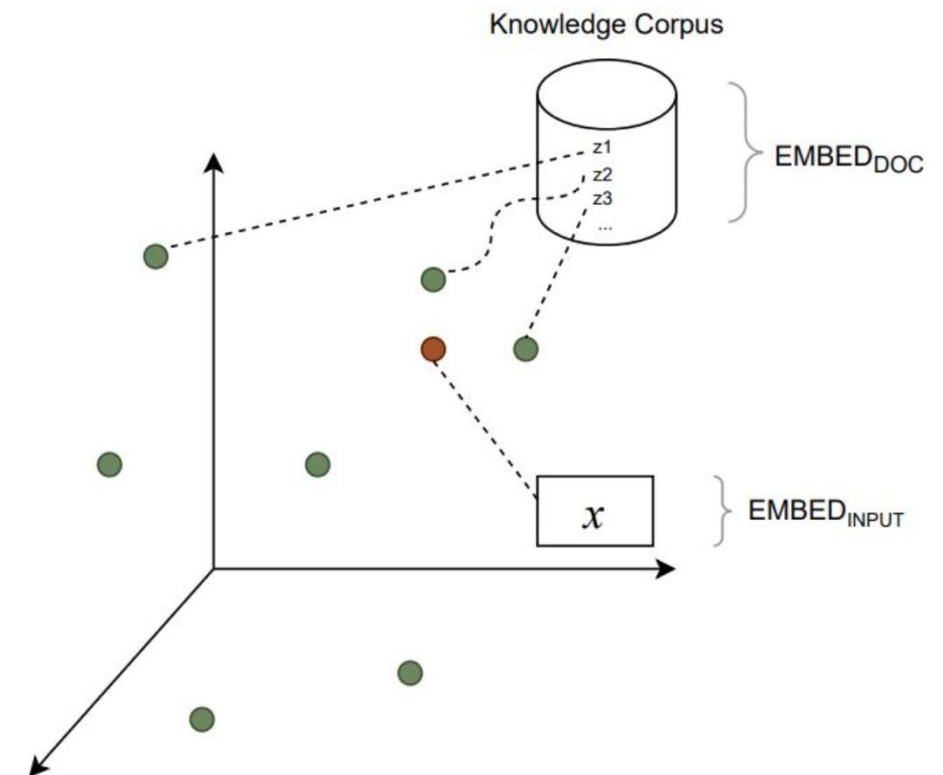


Real input to LLM

(Original Input) You are a helpful AI assistant. Can you tell me the latest ranking of UIUC?

(Retrieved Doc) The University of Illinois Urbana-Champaign (UIUC, U. of I., Illinois, or University of Illinois) is a public land-grant research university...

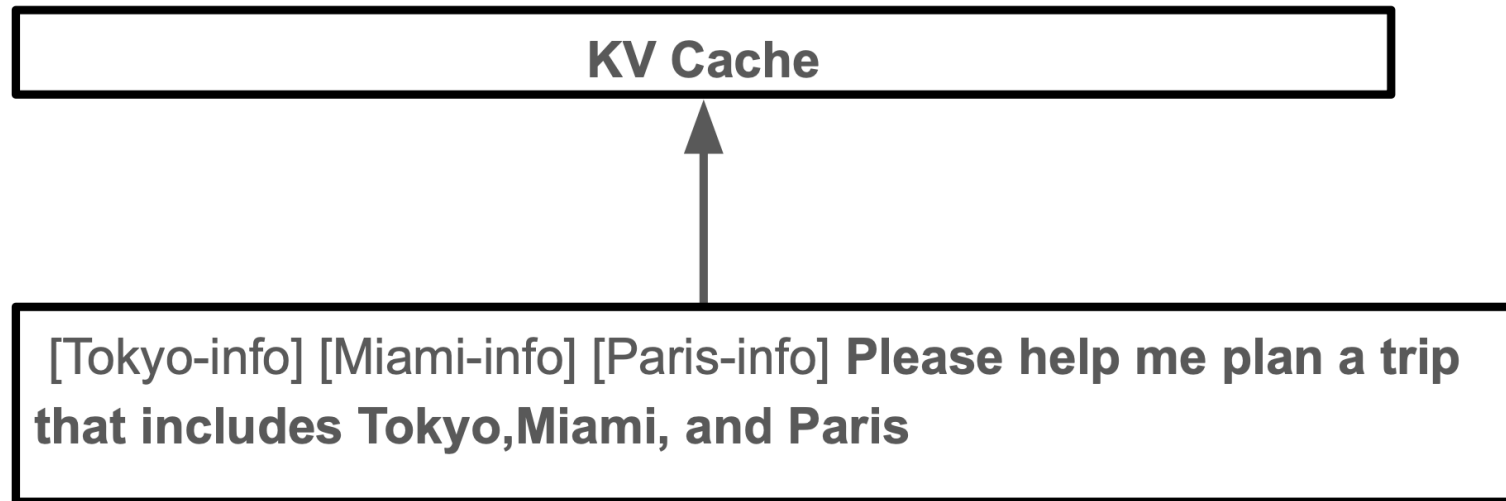
(LLM Response) UIUC is ...



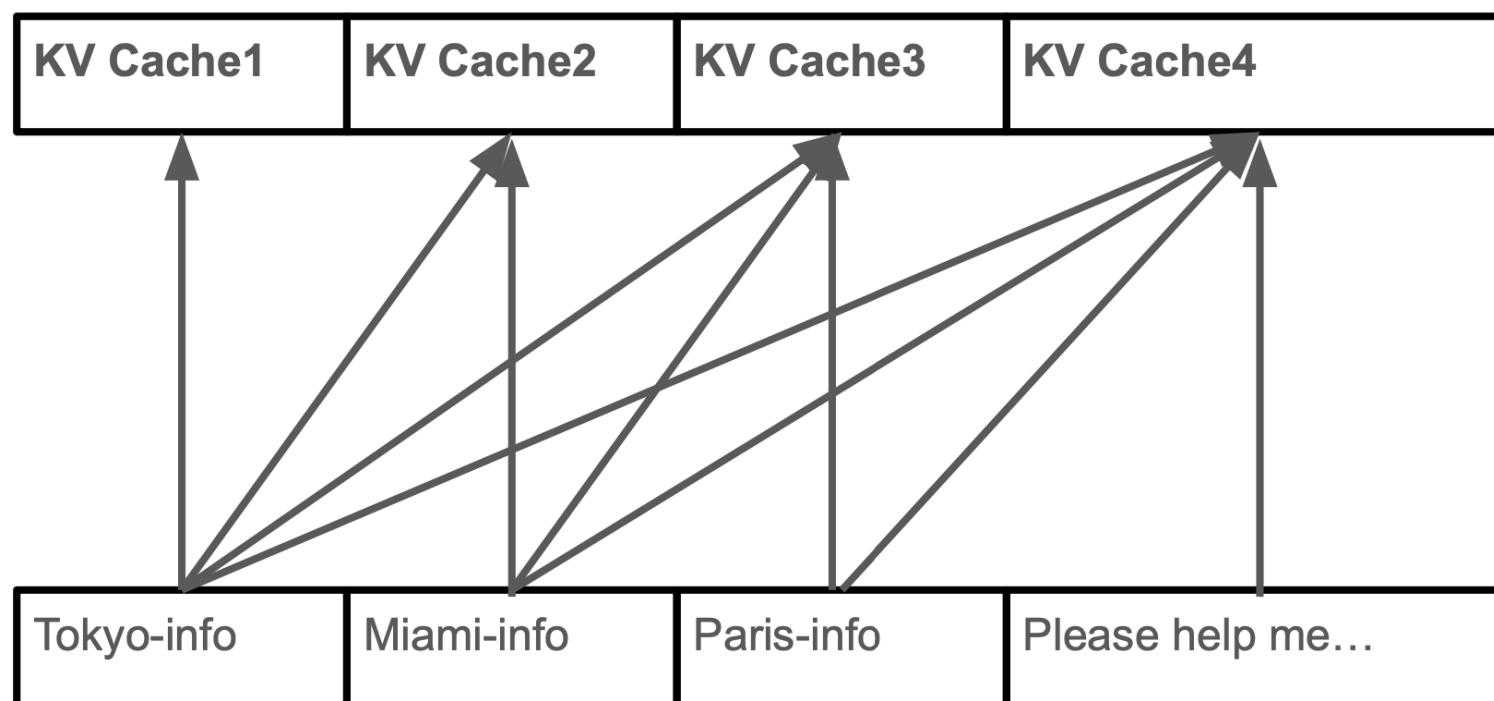
CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion (EuroSys'25, best paper)

- **Key Insight:** cache the KV cache of knowledge and fuse them on the fly with selective token recomputation

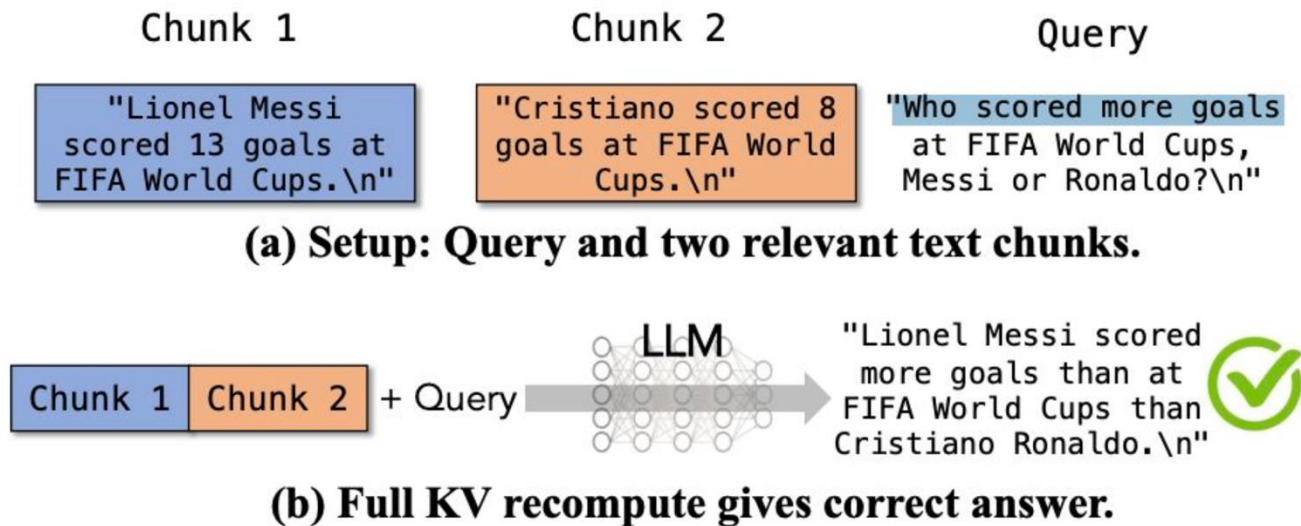
LLM go through the entire input to produce the KV Cache before generating any token.



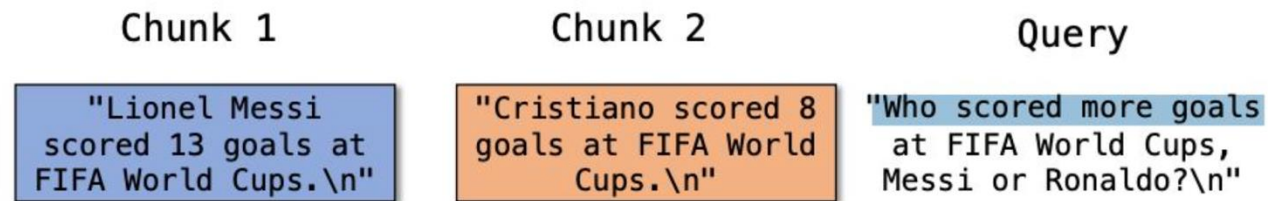
Autoregressive generation: subsequent tokens need to account for preceding tokens



Autoregressive generation: subsequent tokens need to account for preceding tokens



Autoregressive generation: subsequent tokens need to account for preceding tokens



(a) Setup: Query and two relevant text chunks.



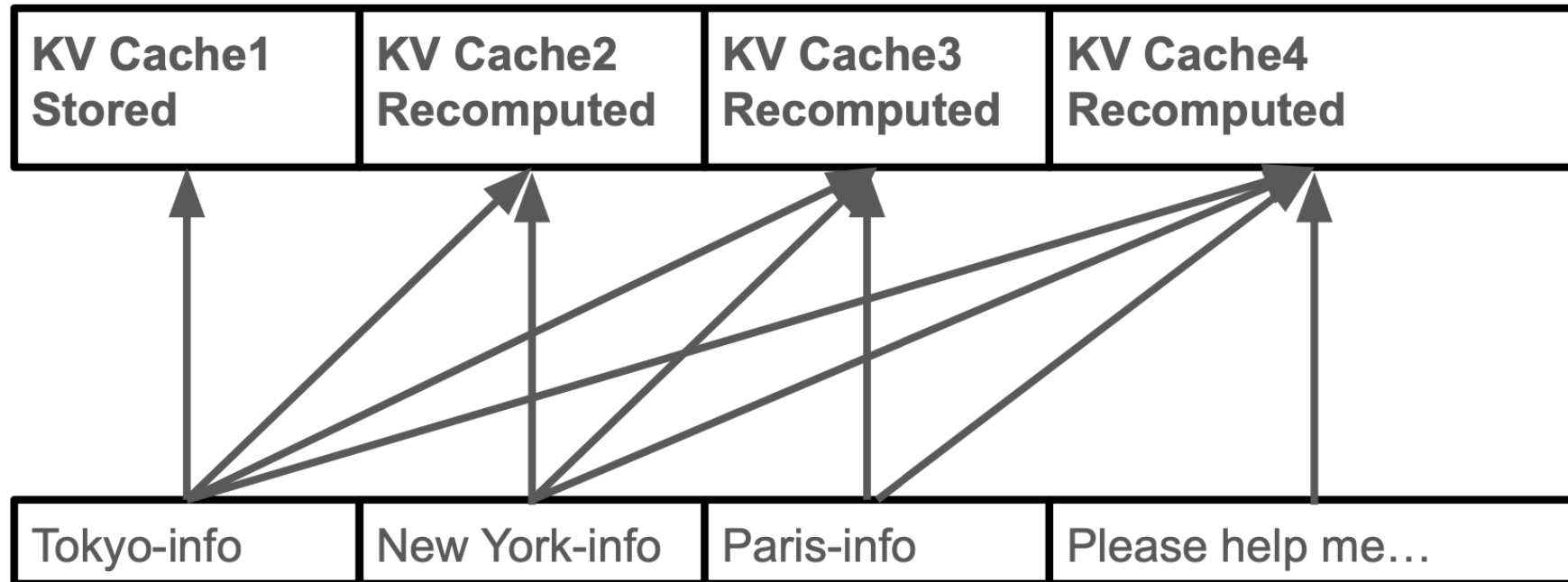
(b) Full KV recompute gives correct answer.



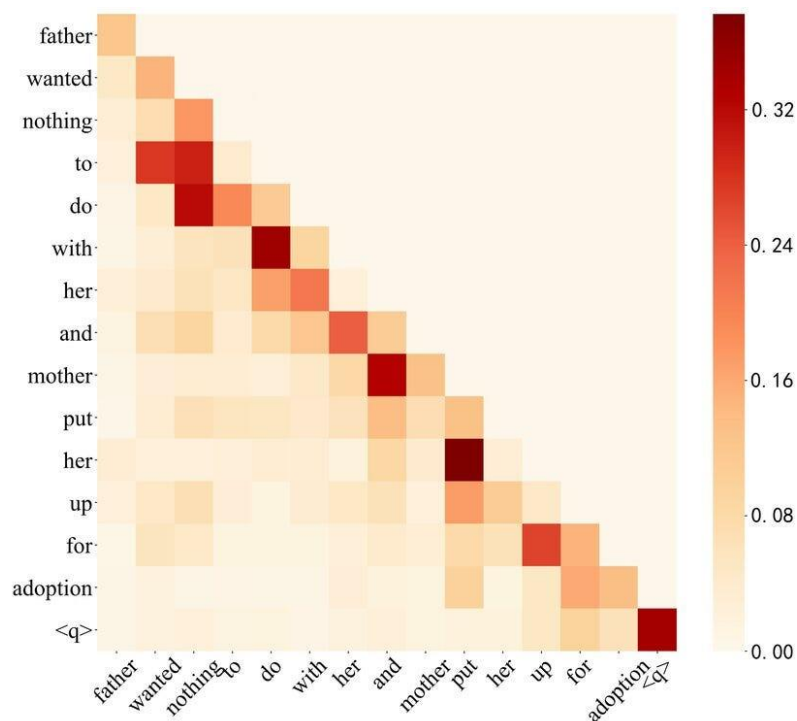
(c) Full KV reuse gives wrong answer.

Autoregressive generation: subsequent tokens need to account for preceding tokens

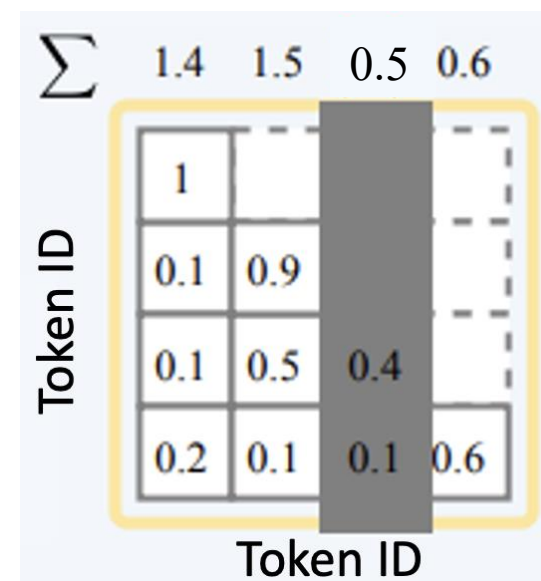
Only reuse the first chunk's KV cache?



Key Observation: not all tokens are equally important, so we can recompute the KV cache for important ones while directly reusing the rest

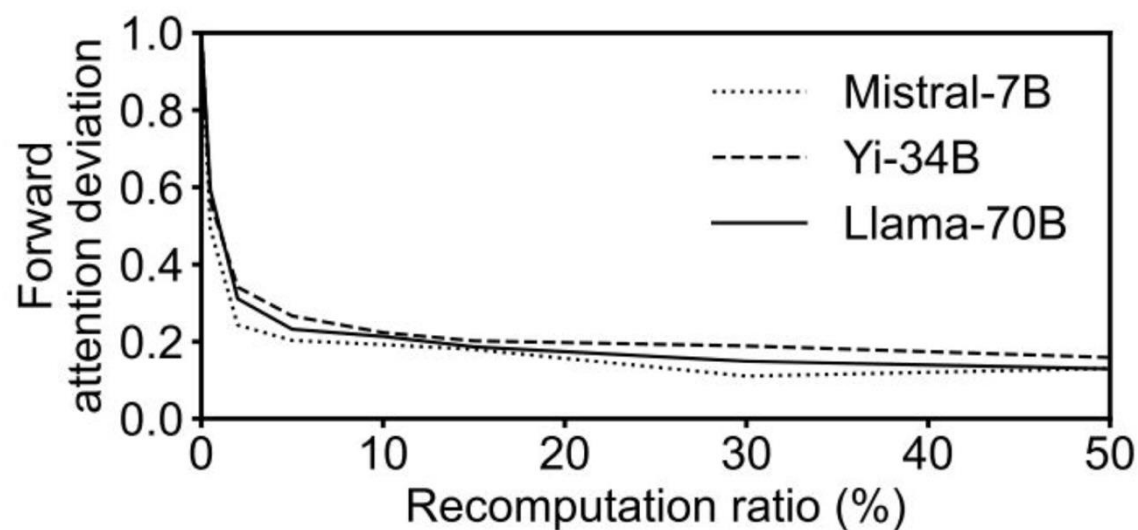
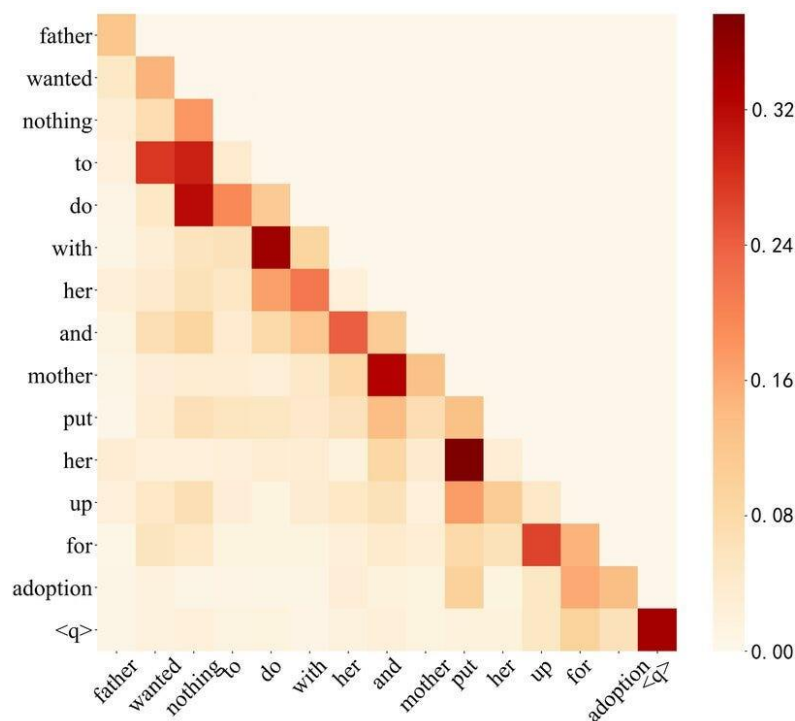


Attention map of tokens

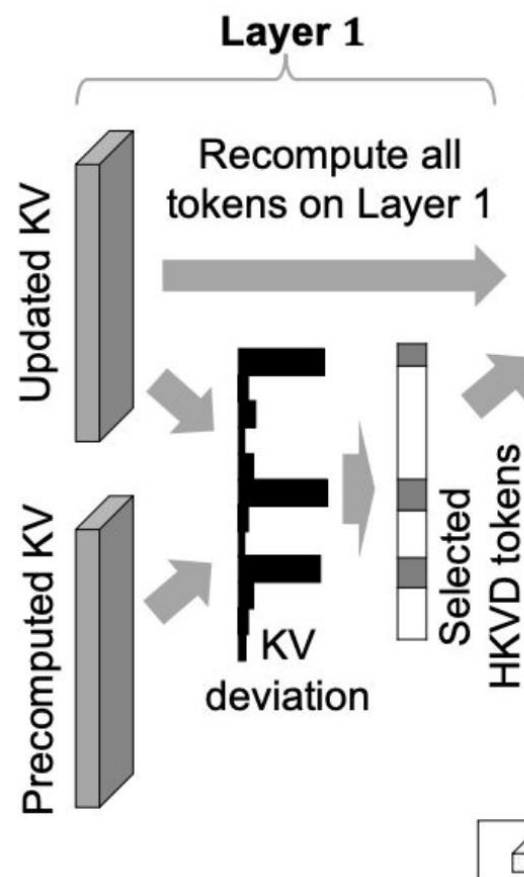


H2O: accumulated attention score

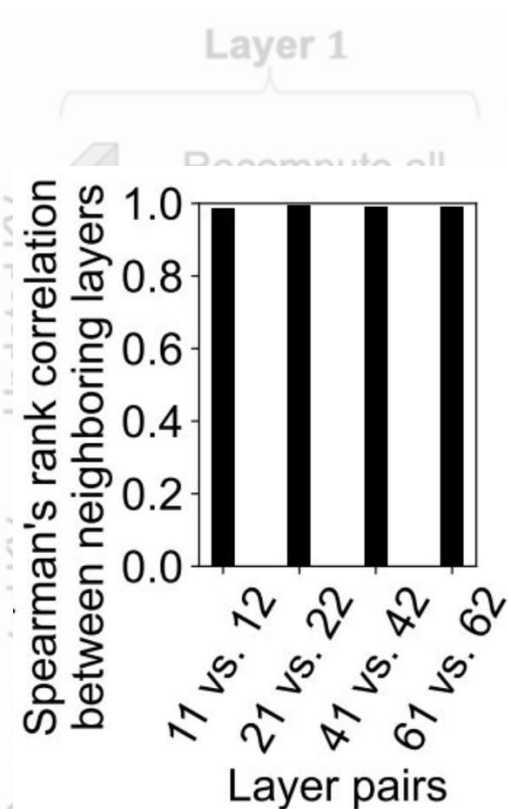
Key Observation: not all tokens are equally important, so we can recompute the KV cache for important ones while directly reusing the rest



CacheBlend with Selective Recomputation



Identifying important tokens by comparing the KV cache deviation (reuse vs. full recomp) in the first layer

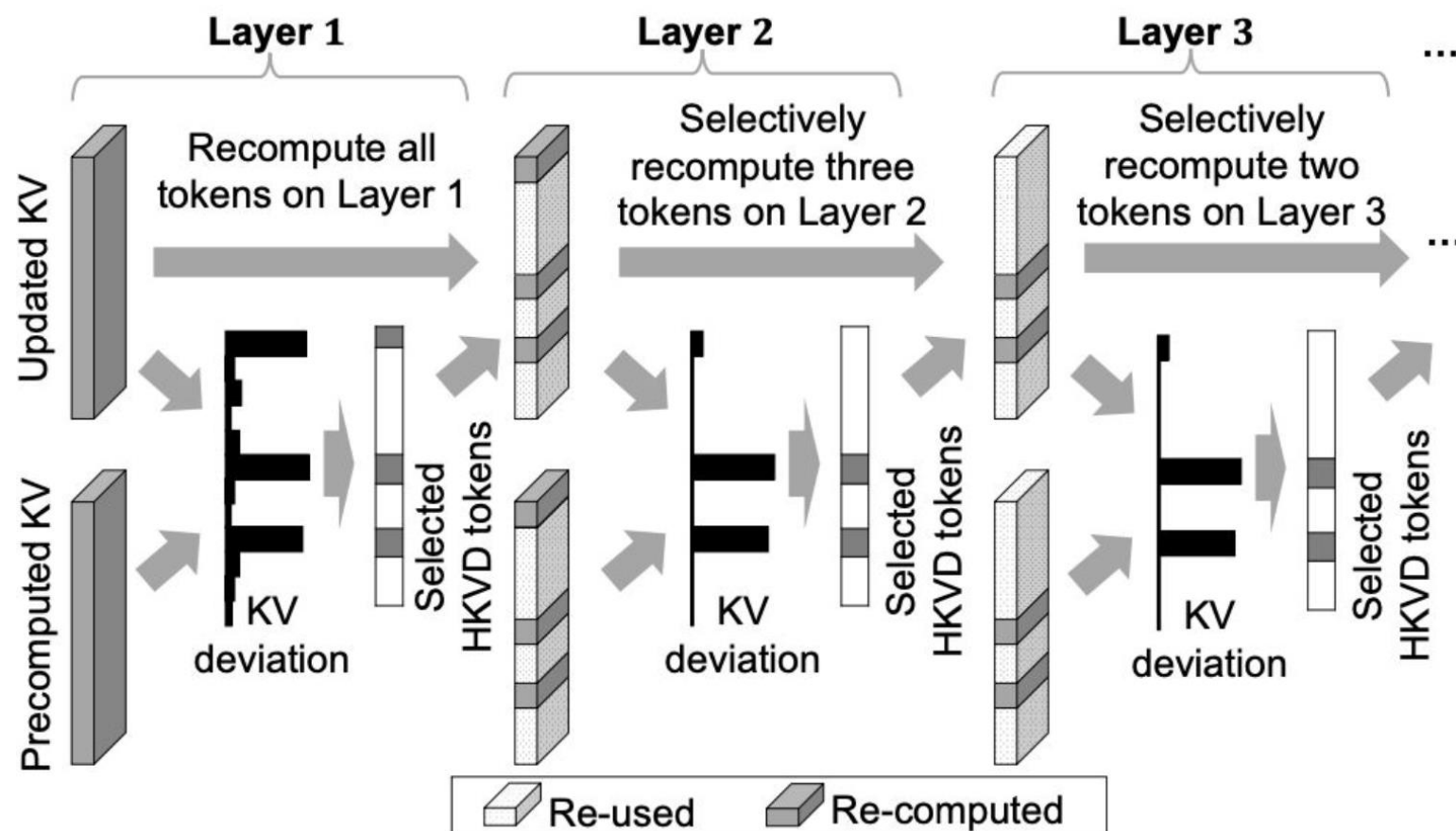


Llama-70B

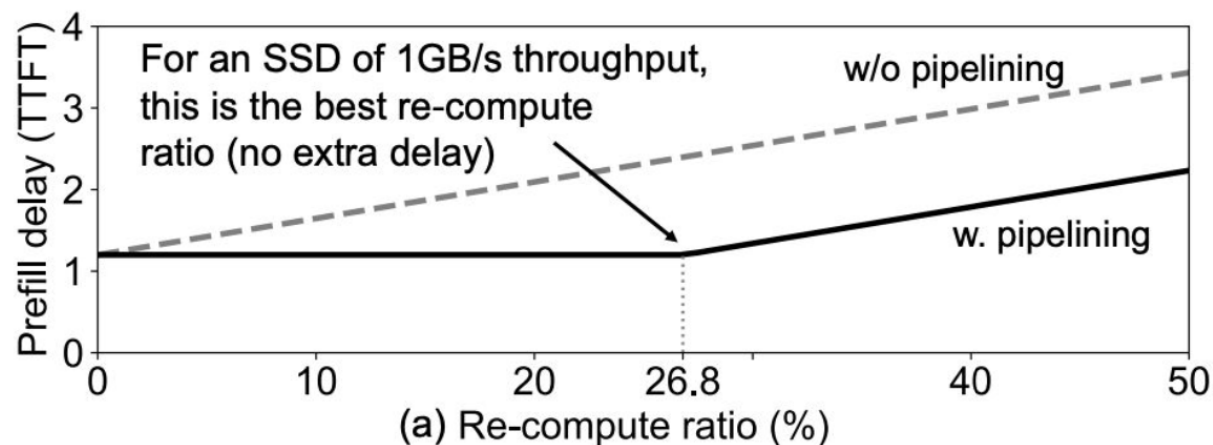
 Re-used  Re-computed

Tokens with the highest KV deviations on one layer are likely to be important on the next layer.

CacheBlend with Selective Recomputation

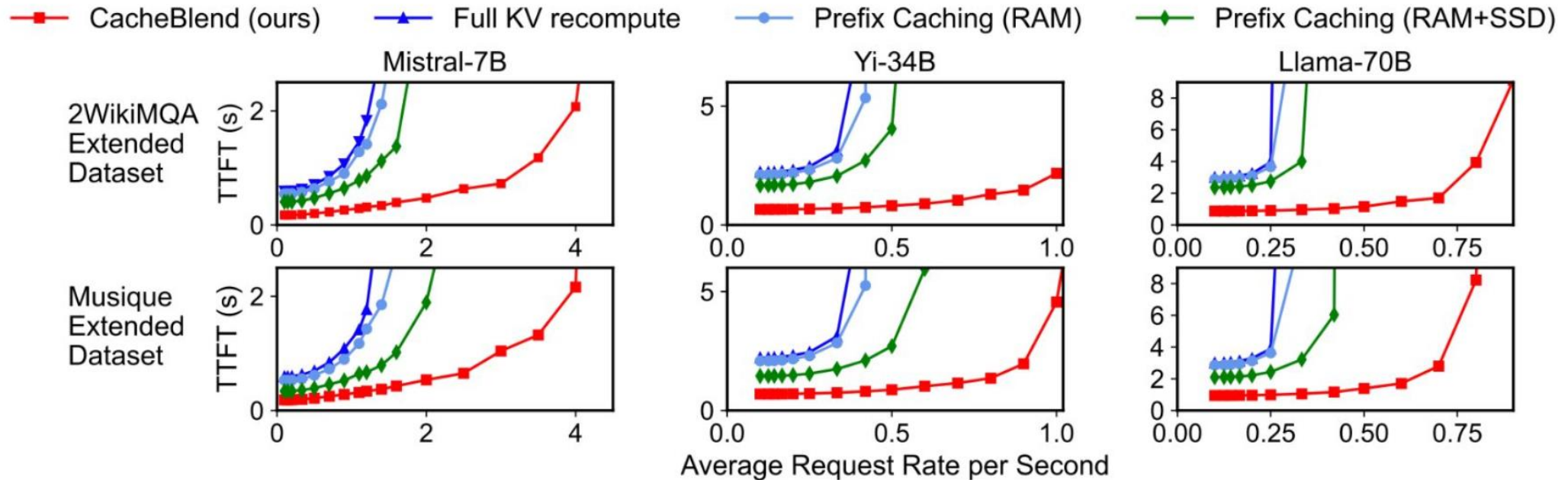


To avoid quality drop, check in every k layers



If the delay for selective KV recompute is faster than the loading of KV into GPU memory, then properly pipelining the selective KV recompute and KV loading makes the extra delay of KV recompute negligible.

Evaluations of CacheBlend



CacheBlend achieves lower TTFT with higher throughput in RAG scenarios compared with baselines of similar quality.

- Generate and Store the KV of different contexts separately
- Concatenate on the fly and identify important tokens based on initial layers
- Selectively recompute KV cache of important tokens

MOONCAKE: Trading More Storage for Less Computation – A KVCache-centric Architecture for Serving LLM Chatbot

Ruoyu Qin^{♠♥1} Zheming Li^{♠1} Weiran He[♠] Jialei Cui[♠] Feng Ren[♥]
Mingxing Zhang^{♥2} Yongwei Wu[♥] Weimin Zheng[♥] Xinran Xu^{♠2}
[♠]Moonshot AI [♥]Tsinghua University

**KV cache-locality aware request
scheduling**

(FAST'25 Best Paper)